

CST-MCP Project Implementation Plan

A comprehensive roadmap for building a localized, synchronous state-managed Model Context Protocol server using Rust and ``rowan`` for Concrete Syntax Tree manipulation.

1. Phased Execution Plan

Phase 1: Core Foundation & CST Engine

- **Objective:** Establish the Rust environment and the foundational ``rowan`` CST parsing logic.
- **Tasks:**
 - Initialize Rust binary project and configure dependencies (``rowan``, ``tokio``, ``notify`` for filesystem watching).
 - Define the grammar and lexer for the target programming language(s) to feed into ``rowan``.
 - Implement the in-memory tree representation to hold loaded files safely.
 - Create unit tests for parsing a file into a red-green tree and losslessly printing it back to a string.

Phase 2: Synchronous State Management & Watcher

- **Objective:** Implement robust file tracking with a synchronized versioned state system to prevent AI edits from colliding with external file modifications.
- **Tasks:**
 - Integrate the ``notify`` crate to build the File Watcher service.
 - Implement ``track(path)`` and ``untrack(path)`` capabilities.
 - Build the state-reconciliation loop: when the watcher detects a change on disk for a tracked file, trigger a synchronous reload to update the in-memory ``rowan`` CST without dropping active edit sessions.
 - Ensure compatibility with standard Linux environments (Debian/Mint) by tuning inotify limits.

Phase 3: MCP Protocol & Tool Exposure

- **Objective:** Wrap the engine in the Model Context Protocol over stdio or SSE.
- **Tasks:**
 - Implement the MCP server lifecycle (Initialize, Ping, Shutdown).
 - Expose ``load_file_to_cst`` and ``save_cst_to_disk`` capabilities.
 - Expose node-level traversal tools (e.g., ``get_tree_skeleton``, ``get_node_by_id``).

Phase 4: Tree Mutation (Edit Capability)

- **Objective:** Enable safe, lossless modifications to the syntax tree.
- **Tasks:**
 - Implement the ``edit_node(node_id, new_content)`` tool.
 - Build the logic to rebuild the ``rowan`` green tree from the modified node upward (preserving the rest of the tree immutably).
 - Implement conflict resolution if a file changes on disk while an edit operation is inflight.

2. Implementation Checklist

Status	Task	Component
[]	Setup Cargo workspace & dependencies (<code>`rowan`</code> , <code>`tokio`</code> , <code>`notify`</code> , <code>`serde_json`</code>).	Environment
[]	Implement language-specific lexer bindings for <code>`rowan`</code> .	Engine
[]	Create <code>`track`</code> and <code>`untrack`</code> functions for adding/removing files from memory.	State
[]	Implement <code>`notify`</code> watcher loop to listen for <code>`Modify`</code> events on tracked files.	Watcher
[]	Build synchronous tree-reload logic for external file changes.	State
[]	Implement <code>`get_node`</code> MCP tool (returns node ID, kind, and text span).	MCP Tools

[]	Implement <code>`edit_node`</code> MCP tool (replaces node and rebuilds red-green tree).	MCP Tools
[]	Implement <code>`save_file`</code> MCP tool (writes lossless tree to disk).	MCP Tools
[]	Write integration tests simulating AI edits alongside background disk writes.	Testing
[]	Test binary deployment on Debian/Linux Mint environments.	Deployment

3. Required Agents, Prompts, and Skills

To successfully orchestrate this development using a multi-agent framework, configure the following specialized agents:

A. The Systems Architect (Lead Agent)

- **Role:** Oversees the synchronous versioned state manager and file watching architecture.
- **Skills:** Rust concurrency (``tokio``), filesystem event handling (``notify``), OS-level file locking, Linux systems programming.
- **System Prompt:** *"You are a Senior Rust Systems Architect. Your primary responsibility is designing a highly stable, local-first state management system for code files. You must ensure that our in-memory CST representations remain perfectly synchronized with the local filesystem on Debian-based systems. Prioritize deterministic behavior, safe concurrency, and absolute data sovereignty."*

B. The CST Engineer (Syntax Agent)

- **Role:** Handles all implementation related to the ``rowan`` crate, parsing, and tree mutations.
- **Skills:** Compilers, Parsers, Lexing, ``rowan`` Red-Green tree architecture, lossless syntax mutation.
- **System Prompt:** *"You are an expert in compiler frontend design and Concrete Syntax Trees, specifically using the Rust ``rowan`` crate. Your task is to map source code into immutable red-green trees, allowing for targeted structural edits without losing a single space or comment of the original formatting. When replacing nodes, ensure the tree rebuilds efficiently."*

C. The Protocol Integrator (MCP Agent)

- **Role:** Bridges the internal Rust engine with the external AI world via the Model Context Protocol.
- **Skills:** JSON-RPC, MCP specification, asynchronous streams, API design.
- **System Prompt:** *"You are the API and Protocol integration expert. Your job is to wrap our local CST engine in a standard Model Context Protocol server. You must expose our capabilities (track, load,*

edit, save) as distinct, well-documented MCP Tools that an LLM can easily understand and chain together."